

HmTest: Automated Testing of HarmonyOS Apps via Model-Driven Navigation and Reinforcement Learning

Yi-Ge Chen^{1, 2} (陈一戈), *Member, CCF, ACM*, Yu-Jia Fan^{1, 2} (樊宇佳), *Member, CCF, ACM*
Si-Nan Wang¹ (王斯南), *Member, CCF, ACM*, Yi-Da Tao² (陶伊达), *Member, CCF, IEEE*
and Ye-Pang Liu^{1, 2, *} (刘烨庞), *Member, CCF, ACM, IEEE*

¹ *Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen 518000 China*

² *Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518000, China*

E-mail: 12011625@mail.sustech.edu.cn; 12431253@mail.sustech.edu.cn; wangsn@mail.sustech.edu.cn; taoyd@sustech.edu.cn
liuypl@sustech.edu.cn

Received December 31, 2024; accepted June 3, 2025.

Abstract HarmonyOS is a new all-scenario operating system for smart devices. As its software ecosystem expands rapidly, how to conduct automated testing of HarmonyOS apps for quality assurance has become a crucial task. This paper presents HmTest, an automated testing framework for HarmonyOS apps, which consists of two complementary modules: targeted exploration and reinforcement learning (RL)-based exploration. Targeted exploration performs white-box testing, leveraging static analysis to construct a page transition graph (PTG). By systematically traversing PTGs to guide testing, HmTest can quickly achieve high page coverages. On the other hand, RL-based exploration performs black-box testing, utilizing reinforcement learning to achieve a comprehensive exploration of app states. Additionally, an automaton-based mechanism is employed to efficiently recover and restart the testing process when reinforcement learning encounters stagnation. We have evaluated HmTest on nine HarmonyOS NEXT apps and compared it with two official HarmonyOS app testing tools. The experimental results demonstrate that targeted exploration can generate highly-precise PTGs and help achieve high page coverages within a few minutes. RL-based exploration can significantly outperform other methods in terms of finer-grained statement coverage on the majority of the tested apps and benefits from the recovery mechanism. To facilitate future research, we have made HmTest open-source at <https://github.com/sqlab-sustech/hmtest> and provided a video demo at <https://jcst.ict.ac.cn/news/361>.

Keywords HarmonyOS app, graphical user interface (GUI) testing, page transition graph, model-based testing, reinforcement learning

1 Introduction

Smart devices such as smartphones and tablets rely on mobile applications (apps for short) to offer users a wide range of functions and have become indispensable in our daily lives. As the graphical user interfaces (GUIs) bridge these apps and their users, GUI testing has been widely employed to test the functionality and reliability of various apps^[1, 2].

HarmonyOS^①, launched as an all-scenario operating system, has rapidly gained market share in recent years and has been installed on millions of smart devices. As of October 2024, there were already over 15 000 native apps and services for HarmonyOS devices^②. Despite the growth and popularity of HarmonyOS apps, the research and development focusing on the GUI testing of these apps is still limited. For example, existing GUI testing tools for Har-

Regular Paper

Special Section of ChinaSoft 2024–Prototype System Competition

The work was supported by the National Natural Science Foundation of China under Grant Nos. 62202213 and 62372219.

*Corresponding Author

①<https://developer.huawei.com/consumer/cn/doc>. May 2025.

②<https://www.globaltimes.cn/page/202410/1321670.shtml>. May 2025.

©Institute of Computing Technology, Chinese Academy of Sciences 2025

monyOS apps, such as wukong^③ and DevEco Testing^④, predominantly rely on black-box testing methods, failing to leverage the internal structure or logic of the apps to achieve comprehensive app testing. As a result, such tools cannot help efficiently and effectively test real-world HarmonyOS apps, especially complex ones.

To address the limitations of existing tools and fill research gaps, we propose a novel and extensible framework named HmTest for automated GUI testing of HarmonyOS apps. Leveraging insights gained from prior work^[3, 4], HmTest integrates both model-based and learning-based approaches to achieve a thorough testing of a given app. To conduct model-based white-box testing, HmTest constructs a page transition graph (PTG), a model specifically tailored for HarmonyOS apps to capture possible transitions between different UI pages in the app, via statically analyzing the underlying ArkTS code. Guided by the PTG, HmTest can perform targeted exploration^[5] following various graph traversal algorithms, such as depth-first search, to quickly maximize page coverages during testing. While such a strategy has its unique value, its traversal of app states is coarse-grained, leaving many complex inter-page transitions and intra-page interactions unexplored. Moreover, it requires access to the apps' source code, which is not always available. To complement the targeted exploration, HmTest also supports black-box app testing by leveraging reinforcement learning (RL) to guide the exploration of the state space of the app under test. To avoid repeatedly visiting a similar set of states (i.e., RL can get stuck in certain scenarios), HmTest further employs an automaton-based mechanism for efficient recovery and restarting of the testing process when necessary. With such a design, the RL-based testing can improve test coverage and achieve a finer-grained exploration of app behaviors, uncovering potential issues that might otherwise remain undetected.

To demonstrate the effectiveness of HmTest, we have evaluated it on nine HarmonyOS NEXT apps, of which eight are open-source, from three aspects: the quality of the constructed PTG, the performance of the targeted exploration, and the enhancement brought by the RL-based exploration. In addition, we have also compared HmTest with baselines. The ex-

perimental results are promising, showing that HmTest can build a high-quality PTG for an open-source app and the two testing strategies can help achieve satisfactory page and statement coverage results (on average, 80.55% and 62.72%, respectively), outperforming the two compared tools on majority of the tested apps.

In summary, this paper makes three contributions.

- We introduce HmTest, a GUI testing tool specifically designed for HarmonyOS apps. The tool is able to extract precise page transition graphs (PTGs) for real-world apps and integrate model-based and reinforcement learning based approaches to enable efficient and effective app testing.

- We evaluate HmTest on nine HarmonyOS NEXT apps and find that it can outperform official tools and achieve high page and statement coverages. This demonstrates that HmTest is a promising solution for comprehensive testing of HarmonyOS apps.

- We make HmTest open-source to foster future innovations (e.g., extending it to support more advanced testing algorithms or AI techniques) and provide practitioners useful testing tools to help ensure the reliability and performance of their HarmonyOS apps^⑤.

A preliminary version of our work was published in the ASE 2024 conference as a tool demonstration paper^[6]. In this journal paper, we significantly extend the earlier version in three aspects: 1) enhancing the PTG construction process by leveraging XPath-based analysis, 2) adding the reinforcement learning based approach to complement the model-based approach and support the GUI testing of closed-source apps, and 3) providing a more comprehensive evaluation of HmTest using apps for the latest HarmonyOS NEXT system, and compare HmTest with two official HarmonyOS app testing tools.

2 Background

2.1 Stage Model of HarmonyOS Apps

HarmonyOS employs a stage model for app development, as illustrated in Fig.1. This model organizes a HarmonyOS app around UIAbility components, which are built-in classes for managing UI pages. Each UIAbility contains a set of ArkUI pages that

^③https://gitee.com/openharmony/ostest_wukong. May 2025.

^④<https://developer.huawei.com/consumer/cn/devco-testing>. May 2025.

^⑤We have made HmTest open-source at <https://github.com/sqlab-sustech/hmtest>, and provided a video demo at <https://youtu.be/HE6ku3Elc2U> and <https://jcst.ict.ac.cn/news/361>.

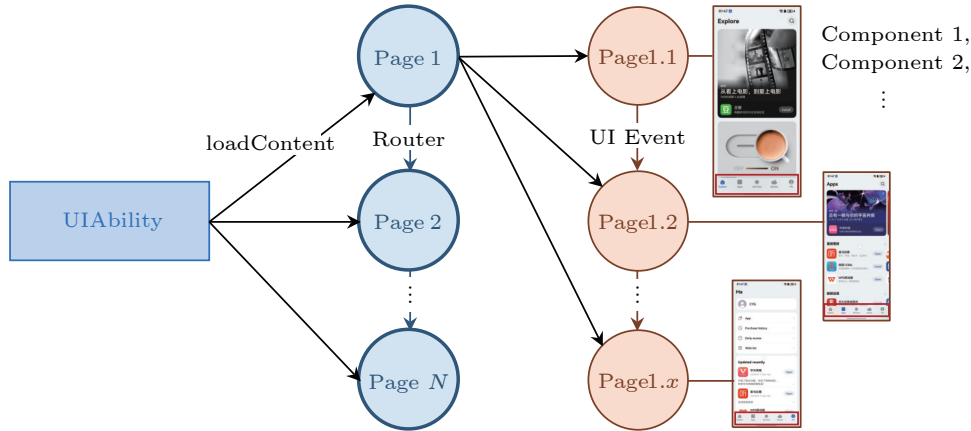


Fig.1. Stage model of HarmonyOS apps.

can transit between each other using the built-in router APIs. Within a single ArkUI page, multiple states can exist due to the variations in its UI layout tree, and the transitions between these states happen dynamically through the execution of UI events. In this paper, we use the term page to refer to an ArkUI page specifically.

2.2 ArkUI

ArkUI is an official UI development framework for building HarmonyOS apps. ArkUI supports two development paradigms: ArkTS-based declarative development and JavaScript-compatible web-like development. The official documentation recommends using the declarative development paradigm for the latest HarmonyOS apps. The declarative development paradigm uses ArkTS, a superset of the TypeScript language with declarative UI syntax, to develop HarmonyOS apps. ArkTS is compiled by ArkCompiler^⑥ to generate bytecode files, which can be executed on the Ark runtime.

3 The HmTest Framework

Fig.2 shows the workflow of HmTest. To perform automatic testing of HarmonyOS apps, HmTest interacts with the apps' GUI by injecting actions to the HarmonyOS devices using hmdriver2^⑦, a UI automation framework for HarmonyOS NEXT apps built on top of the command-line debugging tool hdc. It consists of two main modules, namely, targeted exploration and RL-based exploration.

Targeted exploration conducts white-box testing for HarmonyOS apps when the source code is available. It first uses ArkAnalyzer^[7], a static analyzer for ArkTS, to construct a PTG for an open-source app, which captures the possible transitions between different ArkUI pages. The nodes of the PTG represent coarse-grained states, specifically corresponding to pages within the app. With the high-level guidance provided by the PTG, graph traversal algorithms such as Depth-First Search can then be applied to navigate between app pages efficiently, achieving a high page coverage quickly.

While targeted exploration can help efficiently explore a large number of app pages within a short amount of time, it does not perform fine-grained app state exploration when visiting each page. As such, bugs that arise from complicated user interactions may go unnoticed during testing. To avoid this problem, HmTest also employs RL-based exploration to systematically explore the state space of an app. Specifically, RL-based exploration conducts black-box testing of a HarmonyOS app. It does not require the access to an app's source code (an installation HAP (Harmony ability package) is sufficient for conducting testing). For each interaction, the RL-based exploration module analyzes the UI layout tree of the current app page to select executable components and generate corresponding user actions (e.g., component clicks). If the action involves textual inputs (e.g., when users are expected to fill in an input field), HmTest utilizes a multimodal large language model such as ChatGPT^⑧ to generate meaningful and effective

^⑥https://gitee.com/openharmony/arkcompiler_toolchain. May 2025.

^⑦<https://github.com/codematrixer/hmdriver2>. May 2025.

^⑧<https://chatgpt.com>. May 2025.

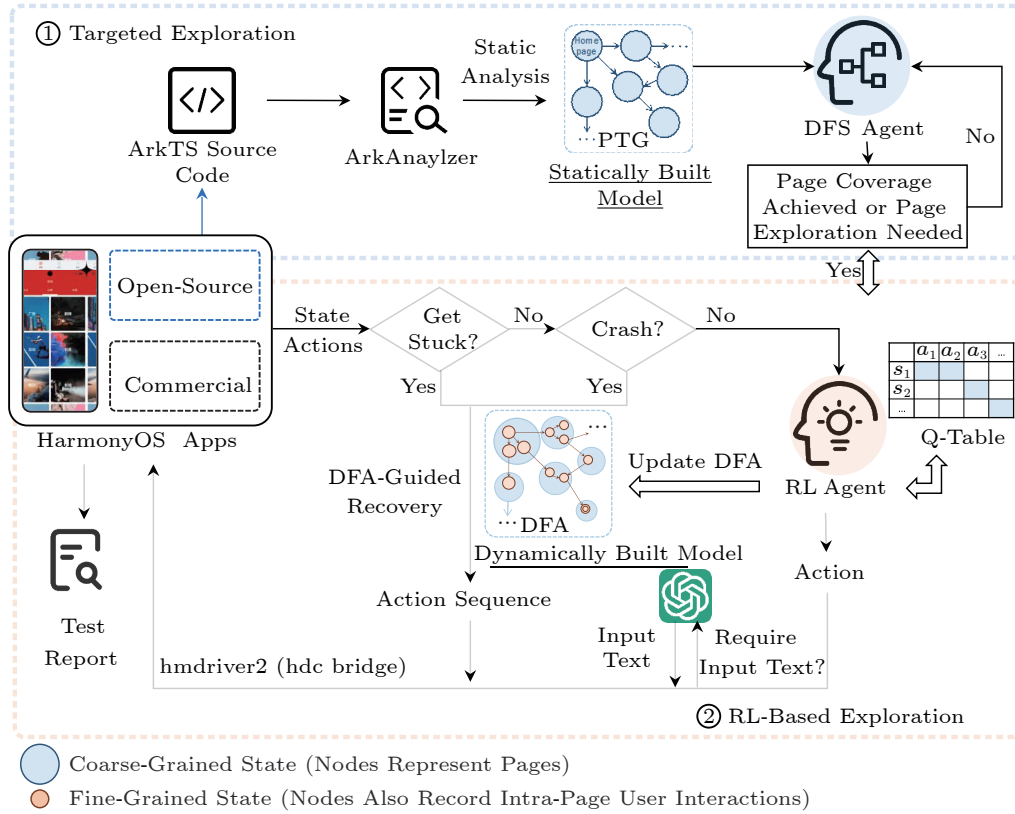


Fig.2. Workflow of HmTest.

tive input content, improving the thoroughness of testing for some input-heavy scenarios^[1]. After performing each action, reinforcement learning algorithms are used to iteratively update the value function or policy, gradually leading to more effective decision-making strategies. In our current implementation, the widely used Q-Learning algorithm has been integrated into HmTest^⑨. To address the issues where the reinforcement learning process might get stuck in loops when exploring certain app states^[8, 9], HmTest further employs a recovery mechanism guided by a deterministic finite automaton (DFA) to avoid the waste of computational resources in such circumstances. The nodes of the DFA represent fine-grained states, corresponding to unique window states in the app. Besides, when the app crashes during testing, the DFA-guided recovery mechanism will also be employed to restart the app exploration while simultaneously logging the action sequence that led to the crashes for bug reproduction and localization.

In the followings, we present the details of the model construction and app exploration processes.

3.1 Building Page Transition Graphs

Page Transition Graph (PTG). A PTG is a GUI model that represents possible page transitions within a HarmonyOS app. Formally, it is a directed graph $G = (V, E)$, where the nodes V represent different ArkUI pages in a HarmonyOS app, and the edges E represent the possible transitions between the app pages represented by the nodes.

To ease understanding, Listing 1 shows a simplified snippet of a HarmonyOS program written in ArkTS from SeaMusic app, and Fig.3 gives its corresponding PTG model. Specifically, the source code in lines 29–31 defines that clicking the self-defined component named `SongListCard` will switch the app from the current page `pages/Collection` to another page `pages/Playing`. To accurately characterize such transitions, we use the XPath^[10] representation to identify specific components on the UI layout tree of the current UI page. The page transition in the example corresponds to the edge from the node `pages/Collection` to the node `pages/Playing`, as illustrated in Fig.3.

^⑨HmTest is an extensible open-source framework. In future, more reinforcement learning algorithms can be integrated into it.

Listing 1. An ArkTS Code Snippet from a HarmonyOS App

```

1  import { router } from '@kit.ArkUI'
2
3  @Entry
4  @Component
5  struct Collection {
6    build() {
7      Column() {
8        Column() {
9          Column() {
10           List() {
11             ListItem() {
12               Grid() {
13                 ForEach([], () => {
14                   GridItem() {
15                     this.TrackItem()
16                   }
17                 })
18               }
19             }
20           }
21         }
22       }
23     }
24   }
25
26   @Builder
27   TrackItem() {
28     Row() {
29       SongListCard().onClick() => {
30         this.handleTrackClick()
31       }
32     }
33   }
34
35   private async handleTrackClick() {
36     router.pushUrl({ url: 'pages/Playing' })
37   }
38 }
39
40 @Component
41 struct SongListCard {
42   build() {
43   }
44 }

```

With the above example, we now present the PTG construction process outlined in [Algorithm 1](#).

Step 1: Initialize. The algorithm begins by performing static analysis on the app source code using ArkAnalyzer to extract the set of classes and methods. Such information serves as the foundation for

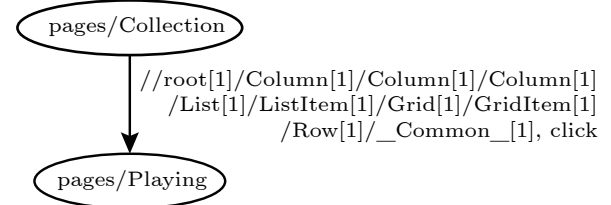


Fig.3. Page transition graph example.

constructing the PTG. An empty PTG is then initialized to represent the page transitions (lines 2–3).

Algorithm 1. PTG Construction

Input: ArkTS source code (C)

Output: PTG

```

1: // Step 1: Initialize
2: scene ← ArkAnalyzer.buildBasicInfo( $C$ )
3:  $PTG \leftarrow \{\}$ 
4: // Step 2: Traverse app pages
5: for class  $\in$  scene.getClasses() do
6:   if class.isEntry() and class.isComponent() do
7:     root ← class.getViewTreeRoot()
8:     treeDFS(root, class.getName(), "root[1]")
9:   end if
10: end for
11: // Step 3: Explore the UI layout tree
12: function treeDFS(node, curPage, nodeXPath):
13:   if node.hasAttribute("onClick") then
14:     method ← node.getOnClickMethod()
15:     callGraphDFS(method, curPage, nodeXPath)
16:   end if
17:   for each child  $\in$  node.children do
18:     childXPath ← deriveComponentName(nodeXPath, child)
19:     treeDFS(child, curPage, childXPath)
20:   end for
21: end function
22: // Step 4: Explore call graph
23: function callGraphDFS(method, curPage, xpath):
24:   if method contains some page navigation related APIs then
25:     targetPage ← extractParameterUrl(method)
26:      $PTG.addEdge((curPage, xpath, targetPage))$ 
27:   end if
28:   callGraph ← scene.makeCallGraph(method)
29:   for each dstMethod  $\in$  callGraph.get(method) do
30:     callGraphDFS(dstMethod, curPage, xpath)
31:   end for
32: end function

```

Step 2: Traverse App Pages. The algorithm iterates through all the classes in the app and identifies entry components, i.e., classes annotated with @Entry and @Component. These classes serve as entry points to different UI pages within the app. For each page, the root node of the tree that represents the layout of the UI components is first obtained using ArkAnalyzer. Then a depth-first search is performed, starting from the root node, to identify all UI components that are possibly related to page naviga-

tion (see step 3 for details).

Step 3: Explore the UI Layout Tree. When traversing the UI layout tree, the `treeDFS` function identifies interactable UI components with `onClick` attributes and retrieves their associated methods (lines 12–21). The primary purpose is to analyze whether a UI component can invoke page navigation related APIs. From our investigation of open-source HarmonyOS apps, click actions are the most common type of interaction associated with page navigation. Therefore, HmTest currently focuses on analyzing `onClick` handlers to capture transitions between pages. While the `onClick` handler often invokes page navigation related APIs directly, there are cases where the navigation logic is implemented through additional function calls. For example, in the code snippet shown in Listing 1, the `TrackItem` method defines a clickable component with an `onClick` handler that invokes the `handleTrackClick` method. In this scenario, the actual navigation logic, i.e., calling the `router.pushUrl` API, is encapsulated within the `handleTrackClick` method. To handle such cases, the algorithm invokes the `callGraphDFS` function to analyze the call graph and resolve indirect function calls that are related to page navigations (see step 4 for more details). Additionally, the `deriveComponentName` function is used to adapt the UI component names identified during static analysis to their actual runtime names, accounting for minor discrepancies introduced during compilation (line 18). For instance, if a self-defined component such as `SongListCard` (line 41 in Listing 1) has some attributes (e.g., `onClick`), the ArkCompiler may wrap it with a `__Common__` component when compiling the ArkTS code to the TypeScript code. The `deriveComponentName` function handles such cases to ensure the accuracy of the PTG.

Step 4: Explore the Call Graph. The methods associated with the clickable UI components identified during the UI layout tree traversal are analyzed with the `callGraphDFS` function. This function detects page navigation related APIs like `router.pushUrl` or `router.replaceUrl`, extracts the target page using `extractParamterUrl` function, and adds the corresponding edges to the PTG (lines 23–27). It also recursively analyzes the call graph of each method to ensure all page transitions are captured (lines 28–31).

3.2 Targeted Exploration

For an open-source HarmonyOS app, the targeted exploration module of HmTest can help maximize page coverages in a short amount of time by utilizing

the PTG constructed for the app. In the following, we present this process, which is outlined in Algorithm 2.

Algorithm 2. Targeted Exploration

Input: *PTG*, *startPage*

```

1: DFS(startPage)
2: explore app systematically (e.g., using model-driven or learning-based strategies)
3: // DFS on PTG
4: function DFS(curPage):
5:   visited.add(curPage)
6:   for each (action, targetPage) ∈ PTG.getEdge(curPage) do
7:     if action exists on current screen and targetPage ∉ visited then
8:       action.execute()
9:       DFS(targetPage)
10:    end if
11:  end for
12:  while there exists (action, targetPage) ∈ PTG.getEdge(curPage) but action not found on current screen do
13:    explore the current page (e.g., using random, model-driven, or learning-based strategies)
14:    if action found and targetPage ∉ visited then
15:      action.execute()
16:      DFS(targetPage)
17:    end if
18:  end while
19:  BackAction().execute()
20: end function

```

Targeted exploration begins with a call to the DFS function (line 1), initiating a depth-first traversal from the initial node (i.e., the node representing *startPage*) of the PTG. During the DFS process, the *curPage* (initially, this is *startPage*) is marked as visited (line 5). Then, the algorithm iterates through all outgoing edges for the node of the current page (line 6). For each such edge, if the corresponding page-switching action exists on the current screen and the target page has not been visited, the action is executed to navigate to the target page (line 8), and DFS is recursively called on the *targetPage* (line 9). If the node of the current page contains outgoing edges but the corresponding page-switching actions are not found on the current screen (e.g., Fig.1) (lines 12–18), the algorithm triggers a search on the current page (line 13). During this phase, if a previously missed page-switching action is identified and its *targetPage* has not yet been visited, the action is executed (line 15), and the DFS traversal resumes from the *targetPage* (line 16). After processing all edges for the current page, the *BackAction* function is invoked (line 19) to return to the previous page.

After completing the deterministic DFS process, which quickly visits different app pages, systematic exploration strategies (line 2) can be employed to al-

low for more fine-grained intra-page user interactions, ensuring comprehensive coverage of app states. For the systematic exploration, various strategies such as model-driven search (e.g., performing a DFS on the UI layout tree of an app page), or learning-based techniques (e.g., using reinforcement learning, as we will show in Subsection 3.3) can be adopted. Such strategies help further explore app states that are not reachable through the deterministic app page traversal process or pages that might have been missed during the PTG construction process (due to the limitations of static analysis).

3.3 RL-Based Exploration

While targeted exploration enables quick page traversals, it cannot achieve a comprehensive coverage of different app states. For this reason, HmTest also employs reinforcement learning to guide the testing system to interact with the app under test in a systematic way. Similar to existing work^[4, 9], we use the model-free Q-learning algorithm in HmTest to allow for systematic app testing. Besides, to facilitate bug diagnosis and enable effective restart of testing when the reinforcement learning process gets stuck or the app crashes, HmTest constructs a deterministic finite automaton to record the executed action sequences and the corresponding app state transitions during testing. In the following, we first formally define DFA and then present the RL-based app exploration process.

Deterministic Finite Automaton (DFA). DFA is a state machine used to model the state transitions of a HarmonyOS app. Formally, it is defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states representing unique window states in the app's GUI, Σ is a finite set of user actions or system events, $\delta : Q \times \Sigma \rightarrow Q$ is the transition function that maps a state and a user action or an system event to a subsequent state, $q_0 \in Q$ is the initial state representing the app's starting GUI state, and $F \subseteq Q$ is a finite set of GUI states where no further transitions are possible.

As shown in Algorithm 3, RL-based exploration begins by launching the HarmonyOS app (line 1) and initializing the DFA and the state set *states* (line 2). The current page is retrieved using the `getCurrentPage` function (line 3), and all executable actions on the page are obtained through `getExecutableActions` (line 4). Based on these actions, an initial state is generated via the `generateState` function (line 5). In

this work, we use the executable action set on the screen to represent the current window state^[11]. The algorithm then enters a loop that continues until the time budget is exhausted (line 25). In each iteration, an action is selected using the `getAction` function (line 7), following the formula below:

$$a_t = \begin{cases} \text{randomly select } a \in \text{actions, with probability } \epsilon, \\ \arg \max_{a \in \text{actions}} Q(s_t, a), & \text{otherwise.} \end{cases}$$

Algorithm 3. RL-Based Exploration

Input: HarmonyOS app *app*

```

1: startApp(app)
2: Initialize DFA A, window state sets states
3: page ← getCurrentPage(app)
4: actions ← getExecutableActions(page)
5: st ← generateState(states, actions)
6: repeat
7:   at ← getAction(st)
8:   at.execute()
9:   page ← getCurrentPage(app)
10:  actions ← getExecutableActions(page)
11:  st+1 ← generateState(states, actions)
12:  rt ← getReward(st, at, st+1)
13:  updateDFA(st, at, st+1)
14:  updatePolicy(st, at, rt, st+1)
15:  if there is no new state for some time then
16:    restartApp(app)
17:    if random() <  $\eta$  then
18:      navigate to the page with minimum access using DFA quickly
19:    end if
20:    page ← getCurrentPage(app)
21:    actions ← getExecutableActions(page)
22:    st+1 ← generateState(states, actions)
23:  end if
24:  st ← st+1
25: until time budget is exhausted
26: // Generate current state
27: function generateState(states, actions):
28:   for st ∈ states do
29:     similarity ← Jaccard(st, actions, actions)
30:     if similarity ≥ threshold then
31:       return st
32:     end if
33:   end for
34:  st+1 ← newState(actions)
35:  states.add(st+1)
36:  return st+1
37: end function

```

Here, a_t represents the selected action at time t , $Q(s_t, a)$ is the Q-value for state s_t and action a , and ϵ is the exploration rate, which helps make a balance

between exploration and exploitation. If a_t is a user action that requires textual inputs, we delegate such operations to a multimodal large language model to generate five different appropriate inputs for the action by providing the current window screenshot with the prompt: “*You are given a screenshot of the page you are browsing, with the cursor focused on a specific input field. Your task is to generate appropriate content for this field. Please return five different generated input texts separated by commas(,) without adding extra spaces around the commas and any explanations.*” We then randomly choose one of the five answers to carry out further testing.

Then the selected action is executed (line 8), and the algorithm retrieves the current page and its executable actions (lines 9–10). In our current implementation of HmTest, we mainly consider five common types of events: click events, long-click events, input events, swipe (scroll) events^[12], and system back events.

During exploration, a new state is generated using the updated actions (line 11). The reward r_t is calculated based on the state transition using the `getReward` function (line 12), following existing work^[9]:

$$\text{getReward}(s_t, a_t, s_{t+1}) = \frac{1}{N(s_t, a_t, s_{t+1})},$$

where $N(s_t, a_t, s_{t+1})$ represents the number of times the transition from state s_t to s_{t+1} via action a_t has been triggered. Such a design encourages the exploration of less frequently visited transitions by assigning higher rewards to them.

Both the DFA and the policy (e.g., Q-table) are updated via `updateDFA` (line 13) and `updatePolicy` (line 14) to refine future action selection. The Q-table is updated based on (1)^[13]:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)), \quad (1)$$

where s_t and a_t are the current state and selected action, respectively, r_t is the reward, s_{t+1} is the next state, α is the learning rate controlling the update step size, and γ is the discount factor determining the importance of future rewards. This formula enables the algorithm to iteratively improve the estimated Q-values by balancing the immediate reward r_t and the maximum expected future reward from the next state.

If no new state is discovered for a preset time period (line 15), the app is restarted using the

`restartApp` function (line 16), and, with a probability of η , the algorithm navigates to the least-explored page based on the DFA (lines 17–19). This approach is adopted to balance exploration and exploitation during the learning process. By allowing the algorithm to occasionally prioritize less-explored states (i.e., setting η to a reasonably high value), it ensures that areas of the app that may have been overlooked during earlier stages of testing can receive sufficient attention, thereby improving overall app state coverage. Conversely, by setting a lower probability for such DFA-guided exploration, the algorithm can maintain the ability to exploit well-known transitions for efficient navigation when necessary. Specifically, the DFA-guided exploration process starts with identifying the state corresponding to the least-covered page. Then, the shortest path from the initial app state to the target state is computed using a breath-first search algorithm on the DFA, which determines the required sequence of actions, $(a_0, a_1, \dots)$. Finally, these actions are executed sequentially to navigate the app to the target state.

Lastly, function `generateState` (lines 27–36) ensures efficient state representation by calculating the Jaccard similarity between the current action set and the action sets of existing states (line 29). If the similarity exceeds a given *threshold* (line 30), the matched state is reused (i.e., the algorithm treats the current state as a previously visited one); otherwise, a new state is created (line 34) and added to *states* (line 35).

4 Evaluation of HmTest

4.1 Experimental Setup

Our evaluation explores three research questions (RQs):

- *RQ1 (Model Quality)*. How is the quality of the PTG model constructed by HmTest?
- *RQ2 (Performance of Targeted Exploration)*. How effective is targeted exploration in achieving page coverage during testing?
- *RQ3 (Performance of RL-Based Exploration)*. Can RL-based exploration effectively boost state-ment coverage during testing?

App Subjects. To investigate the RQs, we first select open-source apps for the latest HarmonyOS NEXT system as our experimental subjects. Currently, there are only a limited number of open-source HarmonyOS NEXT apps available on popular open-source software hosting platforms such as GitHub and

Gitee. For our experiments, we select a total of eight representative apps with non-trivial business logic: six multi-page apps and two single-page apps^⑩. These projects are all developed using the SDK version 5.0.0 (12), which is the minimum API version required to run on HarmonyOS NEXT devices. Table 1 provides the basic information about our selected apps. In addition to open-source apps, we also select one commercial app Weibo, one of the most popular social media platforms in China with a huge number of users. This allows us to evaluate RL-based exploration on a complex real-world app that lacks source code availability.

Tools for Comparisons. To comprehensively evaluate HmTest, we select two official HarmonyOS app testing tools for comparisons: wukong^⑪ (ver. 3.2.0.0) and DevEco^⑫ Testing (ver. 5.0.7.200). Wukong is an open-source command-line tool, offering two primary testing modes: 1) random testing, which simply injects random events to the apps under test and 2) a systematic testing strategy, which performs sequential traversal of UI elements on the discovered app pages. DevEco Testing is a closed-source tool that leverages AI techniques to recognize app UI elements for testing. Due to the lack of available information, the exact UI element recognition and traversal algorithms are unknown.

Evaluation Metrics. For RQ1, to evaluate the quality of the PTG model, we measure the edges in the PTG constructed by HmTest and compare them with the ground truth PTG. Specifically, we compute the precision (P), recall (R), and $F1$ -score ($F1$) for the identified edges according to the equations below. An edge in the PTG is considered a true positive (TP) if the nodes (i.e., pages) and the edge both match the corresponding nodes and edge in the ground truth

PTG, respectively. Otherwise, it is considered a false positive (FP). If the PTG misses an edge available in the ground truth PTG, it is counted as a false negative (FN). The ground truth PTG is constructed through a combination of manual code inspection and validation via dynamic executions. Specifically, we manually examine the code of the open-source apps to identify all router API calls and potential page transitions. Each potential transition ($curPage$, $action$, $targetPage$) is dynamically validated using scripts based on hmdriver2, a tool providing XPath-based component interactions with HarmonyOS NEXT apps, as mentioned in Section 3. For each transition, we first confirm the presence of the current page $curPage$ on the actual device. We then look for the $action$ and execute it via hmdriver2 APIs when it is found (e.g., `driver.xpath("//root[1]/Column[1]").click()`). All transitions that successfully reach the expected $targetPage$ are used to form the ground truth PTG.

$$P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN}, F1 = 2 \times \frac{P \times R}{P + R}.$$

For RQs 2–3, to evaluate the effectiveness of testing, we adopt two coverage metrics: page coverage (PC) and statement coverage (SC). They can provide an intuitive and informative measure of testing thoroughness while balancing runtime overhead for data collection. PC is the ratio of app pages reached during execution to the total number of pages defined in the app. To collect this metric, we use hdc to dynamically retrieve the UI layout tree of the current page during test execution, specifically extracting the `pagePath` attribute. This attribute uniquely identifies the page, and by comparing the pages visited during execution with the full set of defined pages, we are able to calculate the overall page coverage. SC offers

Table 1. Information of the Eight Open-Source HarmonyOS NEXT Apps

HarmonyOS App	#Files	LOC	Star	Fork	#Pages	URL
Wechat_HarmonyOS	36	3 498	434	71	5	https://github.com/ausboyue/Wechat_HarmonyOS
SeaMusic	64	11 945	80	23	8	https://github.com/Okysu/harmony-next-music-sharing
HarmonyHelloWorld	39	5 838	61	7	5	https://github.com/youfrweb/harmonyProject
Handbook	102	6 210	450	95	13	https://github.com/HarmonyOS-Next/interview-handbook-project
MultiConvenientLife	37	5 205	30	21	3	https://gitee.com/harmonyos_samples/multi-convenient-life
MyApplication	33	2 708	18	1	10	https://github.com/Charles-Stark/Youtube-Music-ArkTS-Clone
HMOSWorld	174	17 043	6	1	4	https://github.com/hcgrady2/hmosworld
MultiVideoApplication	56	6 805	33	29	1	https://gitee.com/harmonyos_codelabs/MultiVideoApplication

Note: # means “number of”.

^⑩While the HMOSWorld app contains four pages, we classify it as a single-page application since three of its pages are primarily dedicated to initial login and configuration processes.

^⑪https://gitee.com/openharmony/ostest_wukong. May 2025.

^⑫<https://developer.huawei.com/consumer/cn/deveco-testing>. May 2025.

a finer-grained result. It is defined as the ratio of executed statements to the total number of statements in the app. The information of executed statements was obtained by instrumenting the open-source apps using hvigorw^⑬ that internally employs nyc^⑭ for code instrumentation. It is worth mentioning that due to technical limitations of hvigorw, statement coverage data can only be collected after closing the app under test. This prevents real-time monitoring of statement coverage progression during testing, restricting us to only obtain the final statement coverage when a testing session ends.

Environment & Configurations. The experiments are conducted on a MacBook Pro with an Apple M4 Pro chip (14-core CPU, 20-core GPU, 24 GB unified memory) running Python 3.10.11, connected via hdc to a Huawei Mate60 smartphone running HarmonyOS NEXT 5.0.0, with API version of 5.0.2(14). For the RL-based exploration phase, the learning rate α is set to 0.1 to ensure gradual updates to the Q-values based on observed rewards. The discount factor γ is set to 0.5, reflecting a balanced consideration of both immediate and future rewards during learning. The exploration rate ϵ is set to 0.1, encouraging the algorithm to explore new actions while primarily relying on the learned policy. The initial Q-values are uniformly set to 10¹⁴.

We conduct comprehensive experiments on the eight selected open-source apps, evaluating the four exploration strategies in HmTest, namely, Random, targeted exploration, RL-based exploration without recovery, and the complete RL-based exploration. In addition, we run wukong (both of the two primary testing modes) and DevEco Testing on the open-source apps. Each strategy (or method) is executed for 10 minutes and repeated 10 times on each tested app. It is worth explaining that for targeted exploration, we only focus on the six multi-page apps, as this approach requires multiple pages to be meaningful (i.e., a PTG with a single node cannot provide any useful guidance during testing). Besides, considering the dynamic nature of app pages, we implement a fallback mechanism (lines 1–2 in Algorithm 2): if targeted exploration fails to reach all targeted pages within 45 actions (approximately 6 minutes; the exact number is configurable), the system will automatically switch to RL-based exploration to reach more pages, rather than blindly maximizing page coverage.

For open-source apps, we collect both statement coverage and page coverage metrics for all test sessions of DevEco Testing and the strategies implemented in HmTest. However, due to technical limitations, we are unable to collect real-time page coverages data during wukong’s test sessions and therefore we only report the statement coverage results. To mitigate the effect of randomness, all test coverage results are obtained by calculating the average across all 10 runs.

For the commercial app Weibo, due to the unavailability of source code, we only conduct a single 30-minute test session to evaluate the performance of HmTest by measuring the number of different app states it encountered. As different tools have different app state representations, we cannot run experiments on Weibo to fairly compare HmTest with the two official HarmonyOS app testing tools for now. In the future, we will explore rigorous ways to compare the performance of HmTest and other tools on commercial apps without source code.

4.2 Quality of PTGs

Fig.4 shows an example PTG generated by HmTest for the SeaMusic app. In this graph, the 16 solid lines represent correctly detected edges (TPs), while the two dashed lines indicate missed edges (FNs). The edges in this figure are simplified using the types of components, while the actual edges are represented using the XPath paths of components. The overall results for RQ1 are in Table 2.

For precision, all apps demonstrate promising results. We do not observe any detected edges that are false positives. For recall, the results are perfect for four of the six multi-page open-source apps but show variability in the other two apps. Notably, the handbook app exhibits a significantly lower recall (only 58.62%). The low recall rates can be attributed to two major factors. First, our current analysis only focuses on onClick event handlers that explicitly call router APIs, while implicit page navigation logic embedded in component properties is not fully considered yet. Second, there are certain complex scenarios, such as navigation API parameters being passed through multiple layers of function calls, which the static analysis struggles to handle effectively. These limitations reduce HmTest’s ability to capture all pos-

^⑬<https://developer.huawei.com/consumer/cn/doc/harmonyos-guides/ide-hvigor-commandline>. May 2025.

^⑭<https://github.com/istanbuljs/nyc>. May 2025.

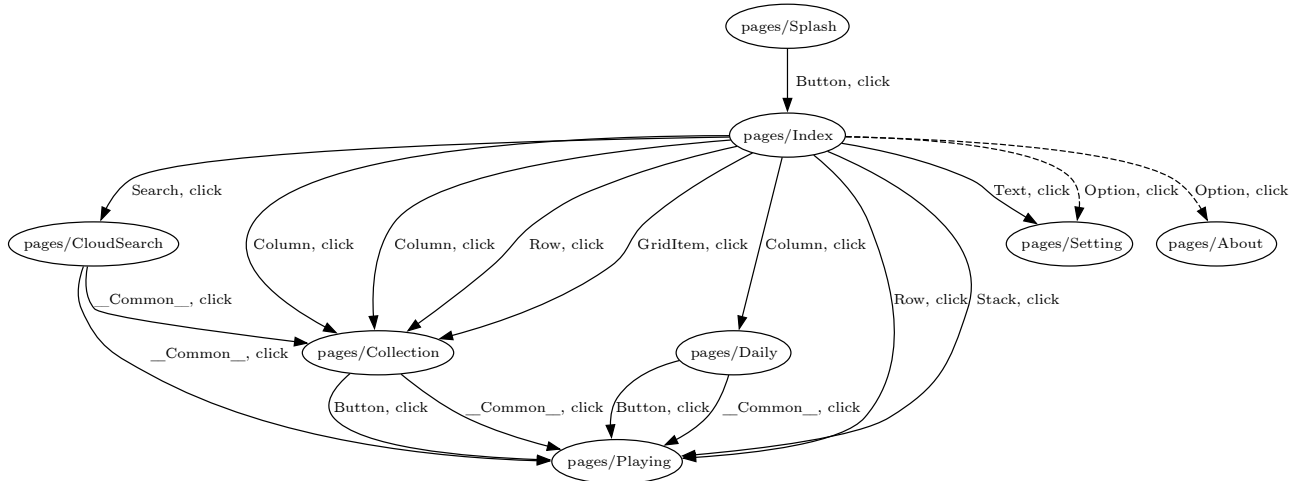


Fig.4. Example simplified PTG of SeaMusic (back actions are omitted in the figure).

Table 2. Quality Evaluation Metrics of PTGs in the Six Multi-Page Apps

HarmonyOS App	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
Wechat_HarmonyOS	0	0	0	100.00	100.00	100.00
SeaMusic	16	0	2	100.00	88.89	94.12
HarmonyHelloWorld	0	0	0	100.00	100.00	100.00
Handbook	17	0	12	100.00	58.62	73.91
MultiConvenientLife	3	0	0	100.00	100.00	100.00
MyApplication	21	0	0	100.00	100.00	100.00

sible page transitions accurately. In the future, to address the first limitation, we plan to extend our analysis to collect attribute information from all types of components. This will help us identify components that register router API callbacks not only through explicit `onClick` methods but also via built-in attribute callbacks (e.g., the action attribute in type `MenuElement` components), thereby enabling us to analyze more possible page transitions. For the second limitation, we plan to integrate data flow analysis into our static analysis process. By tracing the paths of parameter passing, we can identify all potential target pages rather than relying solely on AST information, thereby improving the recall of our constructed PTGs.

Answer to RQ1. The static analysis in HmTest results in highly accurate PTGs for most apps, laying a reliable foundation for model-based GUI testing. However, there are still challenges such as implicit navigation logic and complex parameter passing to be addressed to further improve the quality of the statically generated PTGs.

4.3 Performance of Targeted Exploration

Fig.5 shows the results for RQ2. Compared with the random and pure RL-based exploration strategies,

targeted exploration can quickly explore different UI pages by leveraging the PTGs in five out of the six multi-page apps, ultimately achieving higher page coverages. However, for the app handbook, while targeted exploration initially leads in page coverage, its performance falls behind in the later stages. This is because the constructed PTG is incomplete, causing certain pages being inaccessible through the DFS-guided navigation. In comparison, RL-based exploration is capable of dynamically mitigating such limitations and surpasses targeted exploration in terms of page coverages by the end of the testing period.

Compared with the closed-source testing tool DevEco Testing, our targeted exploration demonstrates better performance in terms of page coverage on four out of the six tested apps. While DevEco Testing leverages vision-based techniques to perform semantic analysis of app pages, it is not capable of extracting app internal information, such as those related to page transitions. As a result, it cannot traverse different app pages in a guided and definite manner like our targeted exploration strategy. Nonetheless, on apps where HmTest fails to extract sound and complete PTGs (e.g., handbook) via code analysis, the vision-based approach adopted by DevEco Testing can offer useful information to facilitate app testing. In the future, one promising solution is to combine code

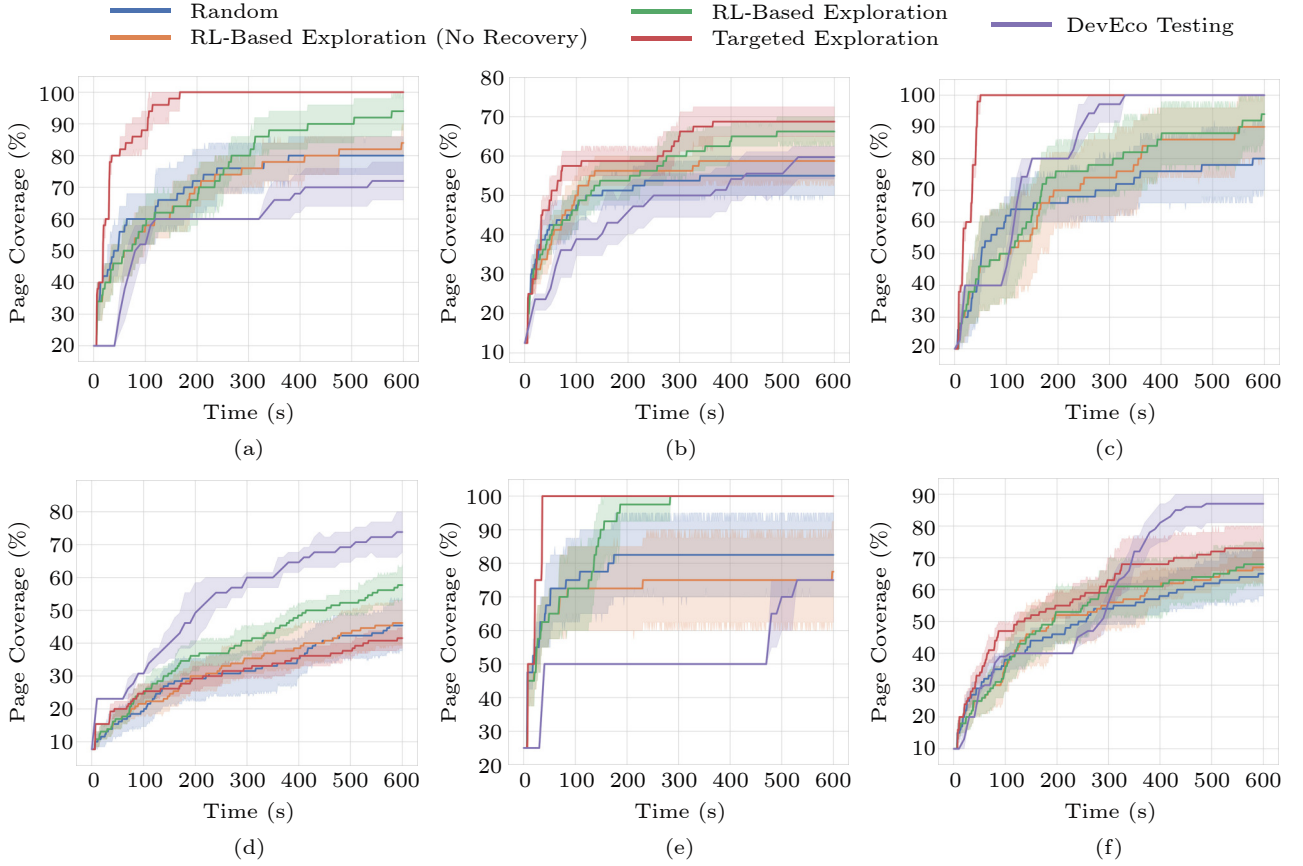


Fig.5. Page coverage results of different methods. (a) WeChat_HarmonyOS. (b) SeaMusic. (c) HarmonyHelloWorld. (d) Handbook. (e) MultiConvenientLife. (f) MyApplication.

analysis based approaches and vision-based approaches to achieve comprehensive app testing.

Despite the limitation caused by inaccurate static analysis, targeted exploration demonstrates clear advantages in the majority of tested open-source apps, achieving significantly higher page coverages within the given time compared with random and RL-based testing in HmTest as well as DevEco Testing.

Answer to RQ2. With the PTGs to provide high-level guidance, targeted exploration can efficiently navigate through different pages in HarmonyOS apps.

This makes it an ideal testing strategy when a high page coverage is expected to be achieved within a short time. Nonetheless, when PTGs are incomplete, its performance could be impacted.

4.4 Performance of RL-Based Exploration

Open-Source Apps. Table 3 shows the test coverages achieved by different methods on open-source apps. We can see that while targeted exploration leads in page coverage (already discussed in RQ2), for

Table 3. Statement Coverage Results of Different Methods

HarmonyOS App	Random (%)	Targeted Exploration (%)	RL-Based Exploration (No Recovery) (%)	RL-Based Exploration (%)	Wukong (%) (Random)	Wukong (%) (Systematic)	DevEco Testing (%)
Wechat_HarmonyOS	53.99	57.08	56.19	57.18	23.65	42.06	48.70
SeaMusic	36.53	39.93	38.76	43.73	25.70	17.10	33.49
HarmonyHelloWorld	74.86	87.35	84.44	87.26	51.09	53.52	92.91
Handbook	57.18	55.23	58.85	62.89	41.07	39.95	57.90
MultiConvenientLife	75.52	83.48	73.00	79.91	40.67	53.69	71.47
MyApplication	65.33	67.54	67.49	68.73	55.04	39.59	77.95
HMOSWorld	39.87	—	40.88	48.60	28.65	22.34	43.89
MultiVideoApplication	51.25	—	51.36	53.49	44.79	44.14	51.41

most tested apps, RL-based exploration demonstrates the best statement coverage at the end of the testing sessions. This is because the RL-based exploration module can systematically trigger both inter-page transitions and intra-page interactions, making them particularly effective in exploring complex app state spaces. Notably, the RL-based exploration strategy is designed to assign higher rewards to unexecuted or less-executed actions as mentioned in Subsection 3.3. Such actions are more likely to explore untested code paths and thus boost the statement coverage. Conversely, repeatedly triggering the same actions typically leads to the execution of similar or identical sets of statements, offering little value in improving statement coverage. This reward-driven mechanism encourages the exploration of diverse interactions, which is a key factor in achieving high statement coverage. Furthermore, the inclusion of the DFA-guided recovery mechanism can also help improve test coverage for RL-based exploration. This improvement is mainly attributed to the ability of the DFA to guide the testing process to escape situations where reinforcement learning gets stuck.

As for the comparison with the two official testing tools, we can see that RL-based exploration consistently outperforms the two testing modes of wukong (i.e., random and systematic, as introduced in Subsection 4.1) in terms of statement coverage across all tested apps. Compared with DevEco Testing, RL-based exploration also achieves higher statement coverages on the majority (6/8) of the tested apps.

Commercial App. Fig.6 presents the experimental result on Weibo. As Weibo adopts the single-page application architecture (i.e., there is only one single ArkUI page in the app) and is closed-source, we cannot use page or statement coverage to compare differ-

ent methods. Instead, we use the dynamically observed state number as a proxy to evaluate test coverage. Fig. 6 shows the number of unique states visited by each method over time¹⁵. As we can clearly see, RL-based exploration significantly outperforms both its variant without DFA-guided recovery and the random strategy, demonstrating its ability to dynamically discover new states even under challenging conditions.

Answer to RQ3. By leveraging reinforcement learning, HmTest can achieve higher statement coverage compared with other methods in both open-source and commercial apps. Our results also confirm that the DFA-based recovery mechanism plays a critical role in resolving stuck states, enabling continued exploration to maximize test coverage.

4.5 Threats to Validity

The validity of our study results may be subject to several potential threats as discussed below.

4.5.1 External Validity

Our evaluation is primarily based on open-source apps, as collecting test coverage information requires instrumentation, which is currently not feasible for closed-source apps due to the lack of reliable tools. This limitation prevents us from conducting experiments on a broader range of commercial apps and faithfully comparing different approaches. Consequently, our findings may not fully generalize to all commercial apps. In the future, we plan to explore alternative methods for measuring test coverage of closed-source apps and expand our evaluation to include a more diverse set of commercial apps to strengthen the generalizability of our results.

4.5.2 Internal Validity

The performance of RL-based exploration is sensitive to hyperparameter selection, such as the learning rate α . To mitigate this threat, we adopt parameter configurations validated in prior RL-based testing research^[14], maintaining consistency across all experiments. While this ensures fairness in comparisons, the parameter configurations may not be optimal for some specific types of apps.

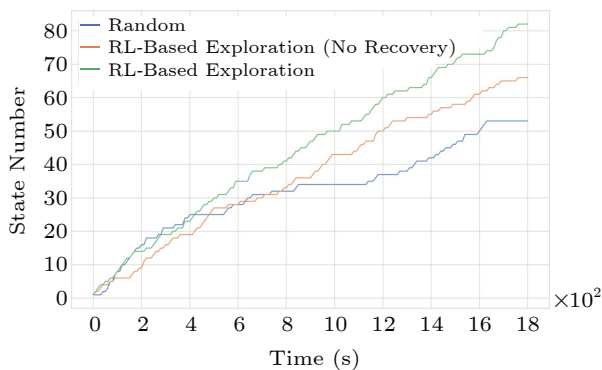


Fig.6. HarmonyOS NEXT commercial app Weibo.

¹⁵Targeted exploration is not applicable as the source code is not available to construct the PTG model.

4.5.3 Construct Validity

In our experiments, statement coverage information of the tested apps is collected using the tool hvigorw, as mentioned earlier. However, due to inherent mechanisms in hvigorw, the collected results may vary depending on how an app is terminated. While our tool, HmTest, stops apps through the task manager, DevEco Testing occasionally terminates apps by sending commands through the hdc debugging bridge. This difference in implementation may prevent hvigorw from fully capturing coverage data in DevEco Testing, potentially affecting the comparability of results. To mitigate this threat, we run DevEco Testing on each tested app for 10 times, aiming to reduce the impact of occasional app terminations via commands. In the future, we plan to collaborate with the developers of hvigorw to improve its robustness in handling app termination and enhance the accuracy of coverage data collection.

5 Related Work

5.1 HarmonyOS App Testing and Analysis

HarmonyOS is in the early stages of ecosystem development with limited existing research. Li *et al.*^[15] outlined a research landscape for HarmonyOS, emphasizing GUI modeling and testing. Unlike Android apps, where each Activity is typically associated with a single XML layout file, one HarmonyOS app employs a single UIAbility to manage multiple ArkUI pages, complicating the analysis of page transitions.

There are several testing tools available for HarmonyOS apps. For example, wukong is an open-source command-line tool in the HarmonyOS NEXT system designed for stability and stress testing, offering features like event injection and user behavior simulation. We include it as a baseline in our experiments. However, wukong cannot provide detailed coverage results (e.g., page coverage), hampering a comprehensive comparison with HmTest and other existing tools. DevEco Testing is another official testing tool that offers streamlined testing functionalities. It provides test reports for issue identification. While it relies on vision-based techniques to perform semantic analysis on app pages to guide the testing process, it cannot leverage app internal information to effectively perform comprehensive exploration of different app states. As revealed in our experiments, the test cover-

ages achieved by DevEco Testing are relatively low, highlighting opportunities for further enhancement.

5.2 Model-Based GUI Testing

Model-based testing methods are widely explored for Android apps, where a navigation model of the app under test is constructed to facilitate the generation of test cases. Yang *et al.*^[16, 17] employed static analysis to create window transition graphs as models, while AndroidRipper^[3] and MobiGUITAR^[18] dynamically build models to abstract the user interfaces. Additionally, A³E^[5] uses activity transition graphs with taint analysis, adopting a DFS strategy for testing. PROMAL^[10] leverages program analysis and machine learning to generate window transition graphs. Different from existing tools, HmTest constructs a PTG model specifically tailored for HarmonyOS apps to facilitate model-based testing.

5.3 RL-Based GUI Testing

RL-based GUI testing involves utilizing reinforcement learning algorithms to guide app exploration and enhance the efficiency and effectiveness of GUI testing. One notable piece of work is Q-testing^[4], which employs the Q-learning algorithm to test Android apps. Moreover, an increasing number of studies are leveraging deep RL algorithms like DQN^[2, 19], DDPG^[20], or SAC^[20, 21] to support more complex GUI testing scenarios.

6 Conclusions

In this work, we presented an automatic GUI testing tool, HmTest, for mobile apps running on HarmonyOS NEXT devices. To efficiently test HarmonyOS apps, HmTest was designed into two modules, targeted exploration and RL-based exploration, employing a dual-strategy approach combining model-driven navigation and reinforcement learning. The targeted exploration module rapidly traverses and covers various app pages, ensuring that the primary pathways of the app under test are quickly assessed. Meanwhile, the RL-based exploration module complements this by diving deeper into the app's state space, providing a more thorough coverage of less obvious states and transitions. This hybrid method optimizes both the speed and depth of testing, thereby enhancing the overall quality and reliability of Har-

monyOS apps. Our experiments on open-source HarmonyOS apps demonstrate that HmTest can achieve satisfactory performance when testing real-world apps, compared with two official HarmonyOS app testing tools. In the future, we plan to integrate more advanced algorithms (e.g., deep reinforcement learning algorithms) and techniques (e.g., multimodal LLMs) into HmTest to further enhance its capability of exploring the UIs of complex HarmonyOS apps. We also plan to extend HmTest to support more types of user interactions to allow a comprehensive testing of real-world HarmonyOS apps.

Conflict of Interest The authors declare that they have no conflict of interest.

References

- [1] Liu Z, Chen C, Wang J, Che X, Huang Y, Hu J, Wang Q. Fill in the blank: Context-aware automated text input generation for mobile GUI testing. In *Proc. the 45th International Conference on Software Engineering*, May 2023, pp.1355–1367. DOI: [10.1109/ICSE48619.2023.00119](https://doi.org/10.1109/ICSE48619.2023.00119).
- [2] Zhao Y, Harrison B, Yu T. DinoDroid: Testing Android apps using deep Q-networks. *ACM Trans. Software Engineering and Methodology*, 2024, 33(5): Article No. 122. DOI: [10.1145/3652150](https://doi.org/10.1145/3652150).
- [3] Amalfitano D, Fasolino A R, Tramontana P, De Carmine S, Memon A M. Using GUI ripping for automated testing of Android applications. In *Proc. the 27th IEEE/ACM International Conference on Automated Software Engineering*, Sept. 2012, pp.258–261. DOI: [10.1145/2351676.2351717](https://doi.org/10.1145/2351676.2351717).
- [4] Pan M, Huang A, Wang G, Zhang T, Li X. Reinforcement learning based curiosity driven testing of Android applications. In *Proc. the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, Jul. 2020, pp.153–164. DOI: [10.1145/3395363.3397354](https://doi.org/10.1145/3395363.3397354).
- [5] Azim T, Neamtiu I. Targeted and depth-first exploration for systematic testing of Android apps. In *Proc. the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, Oct. 2013, pp.641–660. DOI: [10.1145/2509136.2509549](https://doi.org/10.1145/2509136.2509549).
- [6] Chen Y, Wang S, Tao Y, Liu Y. Model-based GUI testing for HarmonyOS apps. In *Proc. the 39th IEEE/ACM International Conference on Automated Software Engineering*, Oct. 27–Nov. 1, 2024, pp.2411–2414. DOI: [10.1145/3691620.3695364](https://doi.org/10.1145/3691620.3695364).
- [7] Chen H, Chen D, Yang Y, Xu L, Gao L, Zhou M, Hu C, Li L. ArkAnalyzer: The static analysis framework for OpenHarmony. arXiv: 2501.05798, 2025. <https://arxiv.org/abs/2501.05798>, Jun. 2025.
- [8] Dulac-Arnold G, Levine N, Mankowitz D J, Li J, Paduraru C, Gowal S, Hester T. Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning*, 2021, 110(9): 2419–2468. DOI: [10.1007/s10994-021-05961-4](https://doi.org/10.1007/s10994-021-05961-4).
- [9] Zheng Y, Liu Y, Xie X, Liu Y, Ma L, Hao J, Liu Y. Automatic web testing using curiosity-driven reinforcement learning. In *Proc. the 43rd International Conference on Software Engineering*, May 2021, pp.423–435. DOI: [10.1109/ICSE43902.2021.00048](https://doi.org/10.1109/ICSE43902.2021.00048).
- [10] Liu C L, Wang H L, Liu T M, Gu D D, Ma Y, Wang H Y, Xiao X S. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proc. the 44th International Conference on Software Engineering*, May 2022, pp.1755–1767. DOI: [10.1145/3510003.3510037](https://doi.org/10.1145/3510003.3510037).
- [11] Baek Y M, Bae D H. Automated model-based Android GUI testing using multi-level GUI comparison criteria. In *Proc. the 31st IEEE/ACM International Conference on Automated Software Engineering*, Sept. 2016, pp.238–249. DOI: [10.1145/2970276.2970313](https://doi.org/10.1145/2970276.2970313).
- [12] Wen H, Li Y, Liu G, Zhao S, Yu T, Li T J J, Jiang S, Liu Y, Zhang Y, Liu Y. AutoDroid: LLM-powered task automation in Android. In *Proc. the 30th Annual International Conference on Mobile Computing and Networking*, Nov. 2024, pp.543–557. DOI: [10.1145/3636534.3649379](https://doi.org/10.1145/3636534.3649379).
- [13] Morales M. Grokking Deep Reinforcement Learning. Manning Publications, 2020.
- [14] Fan Y, Wang S, Fei Z, Qin Y, Li H, Liu Y. Can cooperative multi-agent reinforcement learning boost automatic web testing? An exploratory study. In *Proc. the 39th IEEE/ACM International Conference on Automated Software Engineering*, Oct. 27–Nov. 1, 2024, pp.14–26. DOI: [10.1145/3691620.3694983](https://doi.org/10.1145/3691620.3694983).
- [15] Li L, Gao X, Sun H, Hu C, Sun X, Wang H, Cai H, Su T, Luo X, Bissyande T, Klein J, Grundy J, Xie T, Chen H, Wang H. Software engineering for OpenHarmony: A research roadmap. *ACM Computing Surveys*, 2025. DOI: [10.1145/3720538](https://doi.org/10.1145/3720538). DOI: [10.1145/3720538](https://doi.org/10.1145/3720538).
- [16] Yang S, Yan D, Wu H, Wang Y, Rountev A. Static control-flow analysis of user-driven callbacks in Android applications. In *Proc. the 37th International Conference on Software Engineering - Volume 1*, May 2015, pp.89–99. DOI: [10.1109/ICSE.2015.31](https://doi.org/10.1109/ICSE.2015.31).
- [17] Yang S, Wu H, Zhang H, Wang Y, Swaminathan C, Yan D, Rountev A. Static window transition graphs for Android. *Automated Software Engineering*, 2018, 25(4): 833–873. DOI: [10.1007/s10515-018-0237-6](https://doi.org/10.1007/s10515-018-0237-6).
- [18] Amalfitano D, Fasolino A R, Tramontana P, Ta B D, Memon A M. MobiGUITAR: Automated model-based testing of mobile apps. *IEEE Software*, 2015, 32(5): 53–59. DOI: [10.1109/MS.2014.55](https://doi.org/10.1109/MS.2014.55).
- [19] Lan Y, Lu Y, Li Z, Pan M, Yang W, Zhang T, Li X. Deeply reinforcing Android GUI testing with deep reinforcement learning. In *Proc. the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, Apr. 2024, pp.854–866. DOI: [10.1145/3597503.3623344](https://doi.org/10.1145/3597503.3623344).
- [20] Romdhana A, Merlo A, Ceccato M, Tonella P. Deep reinforcement learning for black-box testing of Android apps. *ACM Trans. Software Engineering and Methodology*

(TOSEM), 2022, 31(4): Article No. 65. DOI: [10.1145/3502868](https://doi.org/10.1145/3502868).

- [21] Du M, Li P, Song L, Chan W K, Jiang B. OAT: An optimized Android testing framework based on reinforcement learning. In *Proc. the 17th International Symposium on Theoretical Aspects of Software Engineering*, Jul. 2023, pp.38–58. DOI: [10.1007/978-3-031-35257-7_3](https://doi.org/10.1007/978-3-031-35257-7_3).



Yi-Ge Chen is a Master student in the Department of Computer Science and Engineering at the Southern University of Science and Technology (SUSTech), Shenzhen. He received his B.S. degree from SUSTech, Shenzhen, in 2024. His research interests mainly include code analysis and software testing.



Yu-Jia Fan is a Ph.D. student in the Department of Computer Science and Engineering at the Southern University of Science and Technology (SUSTech), Shenzhen. She received her M.E. degree from the same department in 2024. Her research focuses on applying AI techniques to automated software testing, especially GUI testing.



Si-Nan Wang received his M.E. degree in computer science and technology from the Southern University of Science and Technology (SUSTech), Shenzhen, in 2020. He is currently a research scholar with the Research Institute of Trustworthy Autonomous Systems of SUSTech, Shenzhen. His main research interest is software quality assurance via software analysis and testing, especially through the lens of software evolution.



Yi-Da Tao is a lecturer in the Department of Computer Science and Engineering at the Southern University of Science and Technology, Shenzhen. She obtained her B.S. degree from Nanjing University, Nanjing, in 2010, and her Ph.D. degree from the Hong Kong University of Science and Technology, Hong Kong. Her research interests include empirical software engineering, AI for software engineering, and software engineering education.



Ye-Pang Liu is a tenured associate professor with the Research Institute of Trustworthy Autonomous Systems (RITAS) and the Department of Computer Science and Engineering at Southern University of Science and Technology (SUSTech), Shenzhen. He leads the Software Quality Lab and is the director of the Trustworthy Software Research Center of RITAS. He obtained his Ph.D. degree from the Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2015, and his B.S. degree with honor from Nanjing University, Nanjing, in 2010. His research interests include software testing and analysis, empirical software engineering, software security, and trustworthy AI. More information can be found on his personal page: <https://yepangliu.github.io/>.